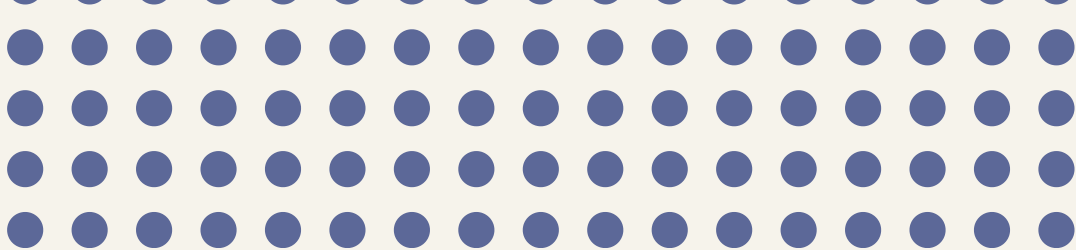


# B VE B+ AĞAÇLARININ GÜNÜMÜZDE KULLANIM ALANLARI

**SIHEM DEMİR**

## 1. Veri Tabanlarında Kullanımı

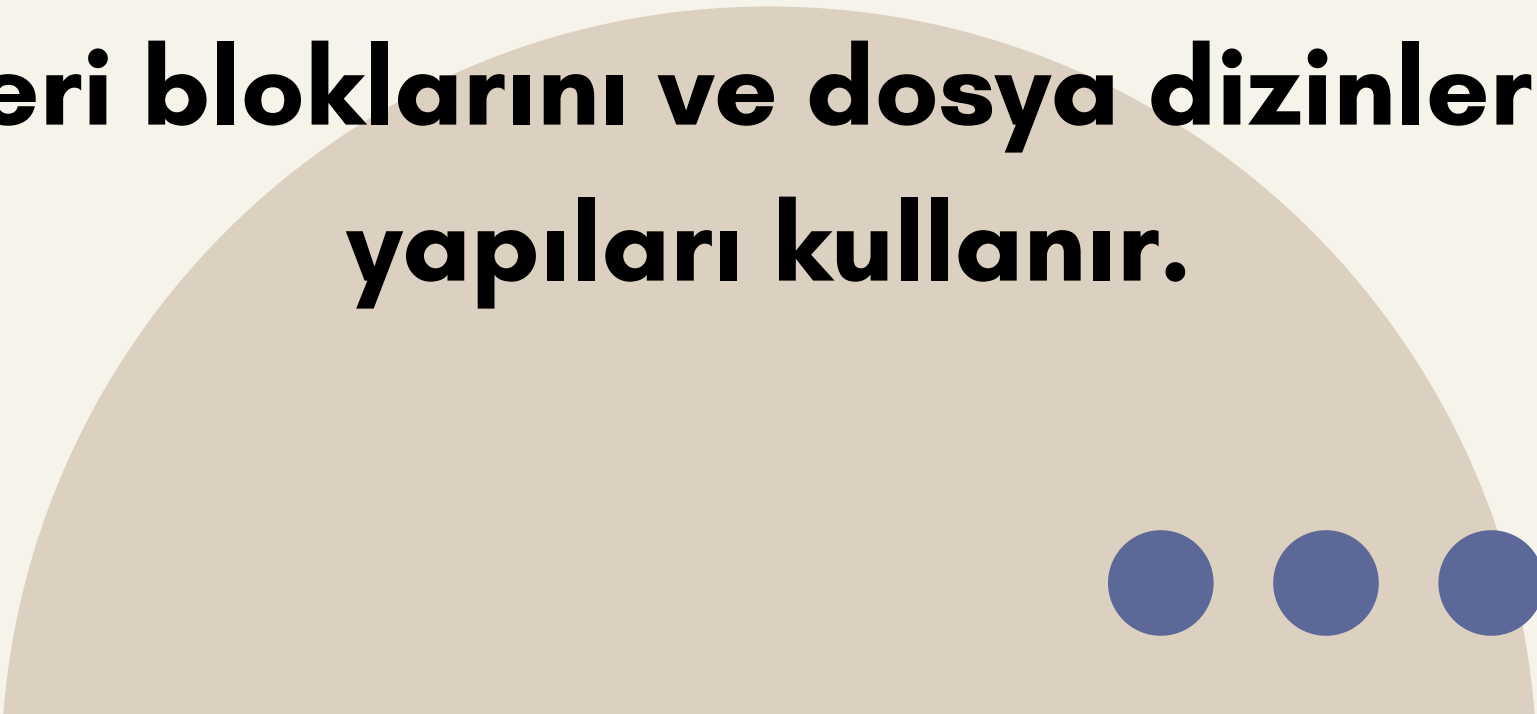
**Veri tabanlarında milyonlarca hatta milyarlarca satır veri bulunur. Bu veriler diskte (HDD/SSD) saklanır. Diskten veri okuma işlemi, RAM'e göre çok yavaştır. B ve B+ ağaçları, veri tabanlarında indeksleme (indexing) mekanizmasının temelini oluşturur. Bu ağaçlar sayesinde veri tabanı, aranan bir kaydı bulmak için tüm tabloyu baştan sona taramak (Full Table Scan) yerine, hedef verinin diskteki tam konumunu nokta atışı tespit eder.**



## 2. Dosya Sistemlerinde Kullanımı

**İşletim sistemleri, bilgisayarınızdaki milyarlarca dosyayı ve klasör yapısını yönetmek için bu ağaçları kullanır. Bir klasörün içine girdiğinizde dosyaların anında listelenmesi, bir dosya adını aratıp saniyeler içinde bulmanız B/B+ ağaçları sayesinde.**

**Örnekler: Linux'ta kullanılan ext4, XFS, Btrfs ve Windows'un temel dosya sistemi olan NTFS veri bloklarını ve dosya dizinlerini yönetmek için bu yapıları kullanır.**



### **3. Arama Motorları ve Büyük Veri Sistemleri**

**Arama motorları (Google, Bing vb.) internetteki milyarlarca sayfayı "Inverted Index" (Ters Yüz Edilmiş İndeks) denilen yapılarla haritalandırır. Bu devasa indekslerin hızlıca sorgulanabilmesi, anahtar kelimelerin disk üzerindeki yerlerinin bulunması ve büyük veri şemalarında (Big Data) verinin bölümlenmesi (partitioning) işlemlerinde B+ ağaçlarından yararlanır.**

## 4. Neden Modern Sistemlerde Tercih Edilmektedir?

**Modern sistemlerin B ve B+ ağaçlarını vazgeçilmez görmesinin temel nedenleri şunlardır:**

**Düşük Disk I/O (Giriş/Çıkış) Sayısı:** Bu ağaçların "dallanma faktörü" (fan-out) çok yüksektir. Yani bir düğüm (node) içinde yüzlerce anahtar tutulabilir. Bu da ağacın boyunun çok kısa (genelde 3-4 katman) kalmasını sağlar. Milyarlarca veri arasından arama yaparken diske sadece 3-4 kez erişim sağlanır.

**Dengeli Yapı (Self-Balancing):** Veri eklendikçe veya silindikçe ağaç kendi kendini dengeler. En kötü senaryoda bile arama, ekleme ve silme işlemlerinin karmaşıklığı hep sabit kalır:  $O(\log n)$ .

**B+ Ağacının Sıralı Erişim Yeteneği:** B+ ağacında tüm yaprak düğümler birbirine bağlıdır. Bu sayede "A ile Z arasındaki tüm verileri getir" gibi aralık sorguları (range queries) inanılmaz hızlı çalışır.





| Sistem Adı     | Kullanılan Mimari | En Önemli Özelliği                                             | Sunum İçin Anahtar Maddeler                                                                                                                                                                                                                                                                                    |
|----------------|-------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MySQL (InnoDB) | B+ Tree           | <b>Clustered Index</b><br><i>(Veri ve indeks bir bütündür)</i> | <ul style="list-style-type: none"> <li>Tablonun kendisi aslında fiziksel olarak bir B+ ağacıdır.</li> <li>Tüm satır verileri en alt katmandaki (yaprak) düğümlerde saklanır.</li> <li><b>Sonuç:</b> Sıralı listelemeler ve aralık sorguları kusursuz çalışır.</li> </ul>                                       |
| PostgreSQL     | B-Tree            | <b>High Concurrency</b><br><i>(Eşzamanlılık optimizasyonu)</i> | <ul style="list-style-type: none"> <li>Standart B-Tree yapısının çok yüksek trafik için modifiye edilmiş halidir.</li> <li>Ağır sorgular altında bile kilitlenme (deadlock) yaşamadan stabil kalır.</li> <li><b>Sonuç:</b> İlişkisel veri tabanları arasında en kararlı indeks performansını sunar.</li> </ul> |
| MongoDB        | B-Tree            | <b>Point Queries</b><br><i>(Nokta atışı arama)</i>             | <ul style="list-style-type: none"> <li>NoSQL (Esnek doküman) dünyasında WiredTiger motoruyla bu yapıyı kullanır.</li> <li>Verileri orta düğümlerde de tuttuğu için tekil dökümanı anında bulur.</li> <li><b>Sonuç:</b> Milyarlarca JSON dokümanı arasından milisaniyede veri çeker.</li> </ul>                 |



# Günlük Hayattan Bir Benzetme: Hastane Arşiv Sistemi

## B-Tree Benzetmesi:

Arşiv odasına giriyorsunuz. Karşınızda katmanlı bir dolap sistemi var.

İlk dolabın çekmecesini açıyorsunuz, üzerinde "M" yazıyor. Çekmecenin içinde hem "Meral Yılmaz" isimli hastanın dosyası (veri) var, hem de altında "A-L arası için sol koridora, N-Z arası için sağ koridora gidin" yazan yönlendirme kartları (pointer) var.

Yani aradığınız hasta o an açtığınız çekmecedeyse işiniz biter. Yoksa yönlendirmeyi takip edip alt dolaplara inersiniz.

Sorun: Dosyalar (veri) her katmana dağıldığı için dolaplarda yönlendirme kartlarına az yer kalır dolaplar çabuk dolar.





## B+ Tree Benzetmesi (Modern Tercih):

Bu sistemde arşiv odası çok daha düzenlidir.

Üst katlardaki dolapların içinde asla gerçek hasta dosyası bulunmaz. Sadece yönlendirme etiketleri vardır (Örn: "A-G arası alt kata", "H-P arası yan odaya"). Dosyalar yer kaplamadığı için bu etiketlerden tek bir çekmeceye binlerce sığdırabilirsiniz (Yüksek dallanma faktörü).

Bütün gerçek hasta dosyaları en alt kattaki (yaprak düğümler) dolaplardadır. \* En muazzam özellik: En alt kattaki tüm çekmeceler birbirine bir ray sistemiyle (linked list) bağlıdır.

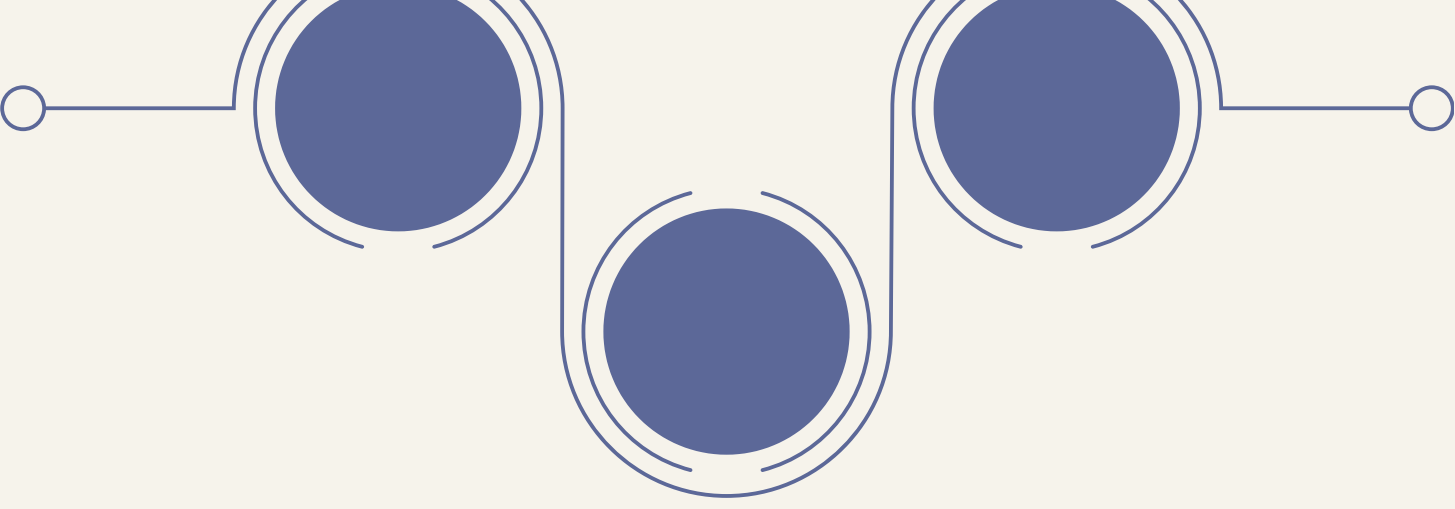
Somut Faydası Nedir? Doktor sizden "Soyadı B ile D arasındaki tüm hastaların geçmişini getir" dedi.

B-Tree olsaydı: Her defasında yukarı çık, etikete bak, aşağı in, dosyayı al, tekrar yukarı çık yapmak zorunda kalırdınız.

B+ Tree sayesinde: Üst katlardaki etiketlere bakarak "B" harfinin olduğu ilk alt çekmeceyi bulursunuz. Sonra yukarıdaki etiketlerle hiç uğraşmadan, alttaki ray sistemini kullanarak "D" harfine kadar yan çekmeceleri kaydıra kaydıra tüm dosyaları hızlıca toplarsınız (Aralık Sorgusu Avantajı).







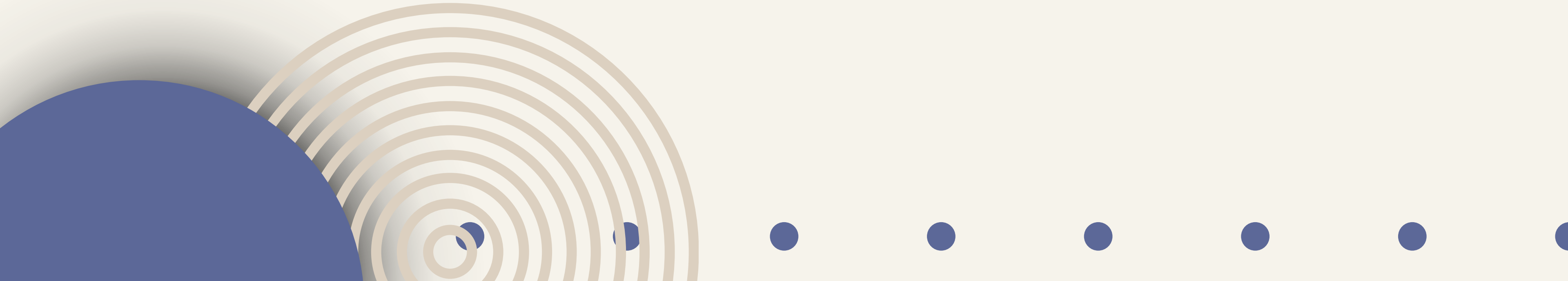
## 1. Veri Yapıları Neden Önemli?

**Bence bu ağaç yapılarının asıl önemi, bilgisayarların o bizim hiç görmediğimiz arka planındaki en büyük sorunu çözmesi. İşlemciler ne kadar hızlı olursa olsun, veriyi hard diskten ya da SSD'den okumak her zaman en yavaş süreçtir. B ve B+ ağaçları olmasaydı, bilgisayar aradığımız bir şeyi bulmak için milyarlarca veriyi tek tek okumak zorunda kalacaktı. Bu ağaçlar sayesinde bilgisayar diske sadece 3-4 kere tıklatarak aradığımız şeyi hemen buluyor.**



## **2. Büyük Veri Çağında Neden İhtiyaç Duyuluyor?**

**Günümüzde veri miktarı artık ıırından ıkmiř durumda. Instagram'daki fotoğraflar, Google'daki aramalar derken her gün petabaytlarca veri üretiliyor. Bu kadar verinin hepsini bilgisayarın hızlı belleğine (RAM) yüklemek hem ok pahalı hem de imkansız. Veri bu kadar devasa olunca, arama yaparken sistemin donmaması gerekiyor. B+ ağalarının en sevdiğim yanı da bu: Veri miktarı 100 katına bile ıksa, arama yaparken harcanan adım sayısı neredeyse hiç deėiřmiyor. Büyük veri çağında sistemlerin ökmeden, tıkr tıkr alışabilmesi için bu yapılara kesinlikle mecburuz.**



### **3. Gelecekte de Kullanılmaya Devam Edecek mi?**

**Bence kesinlikle devam edecek. Tabii ki teknoloji çok hızlı gelişiyor; eski mekanik disklerin yerini artık çok hızlı SSD'ler aldı. Ama mantık yine aynı: Bilgisayarlar veriyi her zaman "bloklar ve sayfalar" halinde okumaya devam edecek. Bu yüzden veriyi gruplayarak saklayan bir hiyerarşiye her zaman ihtiyacımız var. Belki gelecekte yapay zeka destekli akıllı arama sistemleri gelecek, bu ağaçlar biraz daha yapay zekayla hibritleşecek ama temeldeki o B+ ağacı mantığı bence bilgisayar dünyasının merkezinde kalmaya devam edecektir.**



## KAYNAKÇA

Silberschatz, A., Korth, H. F. Veri Tabanı Sistemleri Kavramları. (Ders Kitabı Kaynağı)

MySQL Resmi Dokümantasyonu: "InnoDB İndeksleri ve Çalışma Mantığı"  
([dev.mysql.com](https://dev.mysql.com))

PostgreSQL Resmi Dokümantasyonu: "B-Tree İndeks Biçimleri ve Kararlılık"  
([postgresql.org](https://postgresql.org))

MongoDB Resmi Dokümantasyonu: "WiredTiger Depolama Motoru ve İndeksleme"  
([mongodb.org](https://mongodb.org))

GeeksforGeeks: "Introduction to B-Tree and B+ Tree" ([geeksforgeeks.org](https://geeksforgeeks.org))

Wikipedia: "B-tree" ve "B+ tree" maddeleri ([en.wikipedia.org](https://en.wikipedia.org))

# Teşekkürler

